Computational Physics Methods for Portfolio Risk and Option Pricing

Ali Khabazian

Shahid Beheshti University | Department of Physics

Optimization Methods in Physics Course Report

Supervisor: Prof. S. M. S. Movahed

July 2024

Abstract

This report examines applications of computational physics and numerical optimization methods to financial markets. Monte Carlo simulation is used to generate correlated portfolio returns, estimate value at risk and conditional value at risk, and price European options under risk-neutral dynamics. Antithetic variates and control variates based on option delta and gamma are introduced to investigate the precision and computational cost of simulation estimators. The stochastic foundations of these methods are developed through symmetric random walks, their scaling limit, Brownian motion, and geometric Brownian motion. Finally, loop-based and vectorized implementations of a binomial option-pricing model are compared. The examples illustrate how probabilistic models, numerical sampling, and efficient array operations can support financial analysis. The results remain conditional on the model assumptions and the simulation settings; the reported variance-reduction benchmark also requires verification before a general ranking of the methods can be established.

Keywords: computational finance; Monte Carlo simulation; portfolio risk; variance reduction; stochastic calculus; option pricing

Portfolio simulation with Monte Carlo methods

Monte Carlo methods

Monte Carlo methods use random sampling to estimate quantities that may be difficult to calculate deterministically. In physics, they are particularly useful for systems with many coupled degrees of freedom, including models in statistical mechanics, quantum physics, and the study of complex systems. Randomly generated configurations or trajectories provide a statistical representation of possible system states. Properties of interest are then estimated from averages over those samples.

The same approach can be used to approximate integrals, study stochastic differential equations, and evaluate objectives in numerical optimization. Applications include path-integral formulations in quantum mechanics, the Ising model in statistical mechanics, particle physics, and materials science. In finance, sampling makes it possible to examine the distribution of portfolio values and evaluate contingent payoffs under specified stochastic models [5].

The modern computational form of Monte Carlo simulation was developed in the 1940s through the work of Stanislaw Ulam, John von Neumann, and other researchers at Los Alamos. Its name refers to the casino in Monaco and reflects its reliance on chance and repeated sampling. Ulam's consideration of the probability of winning a solitaire game helped motivate the use of simulated trials to solve more general probability problems. Von Neumann contributed to the mathematical and computational development of the method, including its implementation on early computers. Wider access to computing subsequently extended its use across scientific disciplines.

Monte Carlo simulation is flexible, conceptually straightforward, and applicable to high-dimensional problems. These advantages must be weighed against its computational cost and statistical uncertainty. Increasing the number of independent samples improves precision, but sufficiently accurate estimates may require many simulations. Numerical results should therefore be accompanied by an appropriate assessment of sampling error.

Cholesky decomposition and correlated returns

Cholesky decomposition is a matrix factorization used in finance, engineering, and statistics. A symmetric positive-definite matrix A can be expressed as

$$A = LL^{\mathsf{T}},$$

where L is lower triangular. In financial applications, A commonly represents the covariance matrix of asset returns. The decomposition provides a convenient way to generate multivariate random variables with a prescribed covariance structure.

Let Z be a vector of independent standard normal variables. If L is the Cholesky factor of a covariance matrix Σ, the transformation $Y = LZ$ generates correlated Gaussian variables. This construction is useful in portfolio simulation, stress testing, and risk analysis, where dependence between assets affects the distribution of aggregate returns. It also facilitates linear-system calculations arising in mean–variance portfolio optimization. The Cholesky factors reproduce the specified covariance structure; they do not, by themselves, identify economically interpretable factors.

For the portfolio model considered here, the vector of daily asset returns is assumed to follow a multivariate normal distribution:

$$R_t \sim \mathcal{N}(\mu, \Sigma).$$

The simulation construction is

$$\Sigma = LL^{\mathsf{T}}, \qquad R_t = \mu + LZ_t, \qquad Z_t \sim \mathcal{N}(0, I),$$

where μ is the mean-return vector and I is the identity matrix. This method incorporates the covariance estimated from historical observations into the simulated return distribution. It does not ensure that future market dependence will remain equal to its historical estimate.

Portfolio simulation procedure

Source implementation 1.1.1. The portfolio example uses Python to retrieve historical stock data, estimate return statistics, and simulate possible future portfolio values. The libraries described in the report are pandas, NumPy, Matplotlib, datetime, pandas_datareader, and yfinance.

Two data-retrieval functions are described: one uses pandas_datareader to access Yahoo Finance data, and the other uses yfinance. Both calculate the mean-return vector and covariance matrix. A list of stock symbols is specified, with the appropriate exchange suffix where necessary, such as .AX for Australian securities. The data window starts 300 calendar days before the end date used in the original implementation. Retrieved data are checked for availability before the estimated statistics are displayed.

Random portfolio weights are generated and normalized so that their sum is one. These weights represent the fraction of the initial investment allocated to each asset. The example uses an initial portfolio value of USD 10,000, a simulation horizon of 100 days, and 100 simulated paths. Arrays are allocated to store the mean-return inputs and simulated portfolio values.

For each path, independent normal samples are transformed using the Cholesky factor to produce correlated daily returns. Portfolio returns are calculated from the asset returns and weights, and the portfolio value is updated over successive days. The resulting paths are stored and plotted to show the range of values generated by the assumed model.

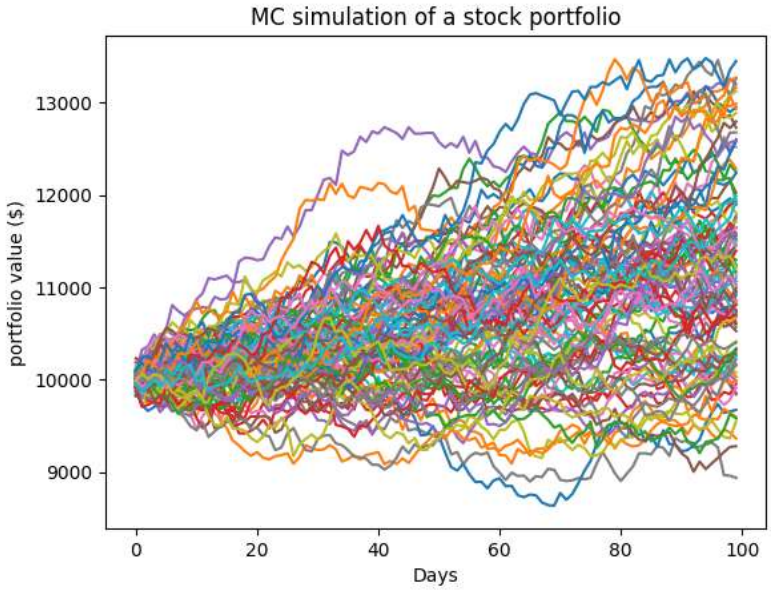


Figure 1. Monte Carlo trajectories for a portfolio with an initial value of USD 10,000 over a 100-day horizon. The figure is retained from source implementation 1.1.1.

The simulation combines historical estimates of mean returns and covariance with randomly assigned portfolio weights. Figure 1 illustrates possible model outcomes rather than a validated forecast of future performance. Cholesky decomposition ensures that the model uses the estimated cross-asset covariance;

the reliability of the resulting scenarios also depends on the data, the Gaussian-return assumption, and the stability of the estimated parameters.

Value at risk and conditional value at risk

Value at risk

Value at risk (VaR) summarizes a specified quantile of the loss distribution over a defined horizon [2]. Its interpretation requires both a confidence level and a time period. Common confidence levels include 95%, 99%, and 99.9%; horizons may be one day, one week, one month, or another period relevant to the analysis.

For example, a one-day VaR of USD 1 million at 95% confidence means that the model assigns a 5% probability to a daily loss exceeding USD 1 million. VaR is a quantile threshold, rather than an absolute upper bound on possible loss. It provides no information about the magnitude of losses beyond that threshold.

Three common estimation approaches are historical simulation, parametric estimation, and Monte Carlo simulation. Historical simulation uses observed market changes to construct an empirical loss distribution. Parametric methods specify a distribution, often a normal distribution, and estimate its parameters from historical returns. Monte Carlo methods generate possible outcomes from a statistical model and calculate the desired loss quantile from the simulated sample.

VaR is attractive because it expresses risk as a single monetary amount that can be compared across portfolios when the horizon and confidence level are consistent. It is used in risk measurement and allocation decisions. Its limitations include sensitivity to model assumptions and historical data, and its inability to describe the severity of losses in the tail beyond the selected quantile. A normal-return assumption, when used, may inadequately represent extreme market movements.

Conditional value at risk

Conditional value at risk (CVaR), also known as expected shortfall or average value at risk, measures average loss in the selected worst tail of the distribution. For a continuous loss distribution at 95% confidence, it is the expected loss conditional on exceeding the 95% VaR threshold. Thus, CVaR addresses the question of how large losses are, on average, within the worst 5% of outcomes.

Historical, parametric, and Monte Carlo methods can all be used to estimate CVaR. Historical and Monte Carlo implementations average the losses in the relevant tail of the observed or simulated distribution. A parametric implementation calculates the tail expectation from the assumed probability distribution. For discrete distributions, the quantile-based definition requires appropriate treatment of probability mass at the threshold.

CVaR provides information about tail severity that VaR alone cannot supply. Under its standard definition, it is a coherent risk measure and satisfies subadditivity: the risk of a combined portfolio does not exceed the sum of the separate portfolio risks. This property is relevant to diversification and portfolio optimization. CVaR is also useful when considering hedging or insurance against adverse market movements.

Estimation can nevertheless be difficult for large portfolios or nonlinear instruments. Because the tail contains relatively few observations, sampling error may be substantial. Errors in the estimated distribution or tail threshold can also affect the resulting CVaR estimate. Both measures should therefore be interpreted in relation to the quality of the underlying data and model.

Monte Carlo estimation of portfolio risk

Source implementation 1.1.2. The risk calculation uses the simulated portfolio array portfolio_sims, whose columns represent individual value trajectories. The final value from each column is extracted into portResults. Two functions then summarize the lower tail of the simulated outcomes.

The function mcVaR(returns, alpha=5) uses NumPy's percentile calculation to obtain the fifth percentile of its input sample. The function mcCVaR(returns, alpha=5) averages values below the corresponding percentile threshold. Here, alpha=5 specifies a 5% lower-tail probability, corresponding to 95% confidence under the loss convention. Although the function arguments are labelled returns, the described calculation applies them to terminal portfolio values. Converting those values to monetary losses requires comparison with the initial investment.

A histogram displays the terminal-value distribution, with vertical lines marking the selected lower-tail value and its conditional average. The source reports a VaR of approximately USD 403 and a CVaR of approximately USD 567 for an initial investment of USD 10,000. The figure displays the more precise labels USD 403.89 and USD 567.44.

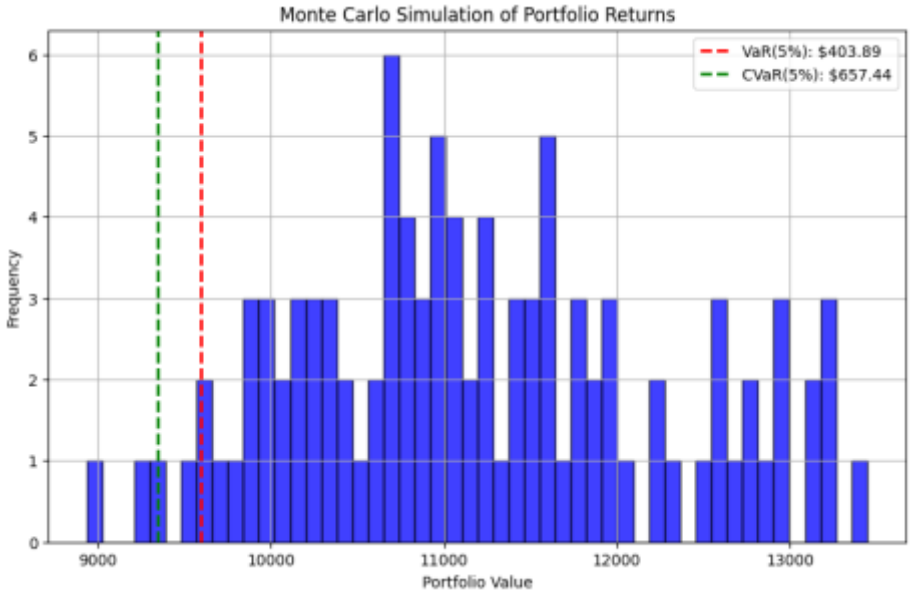


Figure 2. Histogram of simulated terminal portfolio values. The labelled risk estimates are VaR USD 403.89 and CVaR USD 567.44 at a 5% tail probability. The vertical lines indicate the associated portfolio-value levels.

Under the model, the VaR estimate identifies a loss threshold exceeded in approximately the worst 5% of simulations. The CVaR estimate describes average loss in that tail. It does not represent the single worst loss, and VaR does not imply that every tail outcome loses exactly the threshold amount. These distinctions are necessary when interpreting the risk profile produced by the simulation.

Monte Carlo pricing of European options

Options and risk neutral valuation

An option gives its holder the right, without an obligation, to buy or sell an underlying asset at a specified strike price. A call confers the right to buy, while a put confers the right to sell. The option's value depends on the underlying price, strike, time to maturity, volatility, and the risk-free interest rate. Market prices reflect trading and supply–demand conditions, whereas a pricing model estimates value under a specified set of assumptions [1].

Derivative valuation by Monte Carlo simulation uses risk-neutral dynamics. Let C_T denote the derivative payoff at maturity, C_t its value at time t, B_t the value of the money-market account, and $\mathcal{F}_t$ the information available at time t. The valuation relation is

$$C_t = B_t \, \mathbb{E}^{\mathbb{Q}} \left[\frac{C_T}{B_T} \middle| \mathcal{F}_t \right].$$

For a constant continuously compounded risk-free rate r, $B_t = e^{rt}$. The operator $\mathbb{E}^{\mathbb{Q}}$ denotes expectation under the risk-neutral probability measure. The notation $\mathcal{F}_t$ denotes an information filtration, rather than a futures price.

Two different probability models are relevant to the applications in this report. Portfolio statistics, expected returns, VaR, and CVaR concern real-world outcome probabilities. Derivative prices are calculated using risk-neutral probabilities and discounted expected payoffs. Distinguishing these measures prevents historical return estimates from being substituted directly into a risk-neutral pricing calculation.

For a simple European option with an available analytical solution, basic Monte Carlo simulation is computationally less efficient than evaluating that solution. Its value lies in its flexibility for more complex derivatives or higher-dimensional payoffs. Antithetic variates and control variates can improve the precision achieved for a given computational effort.

The Monte Carlo estimator and its standard error

The value of an option is estimated by averaging simulated discounted payoffs. For the ith trajectory,

$$C_{0,i} = \exp\left(- \int_0^T r_s \, ds \right) C_{T,i} = e^{-rT} C_{T,i},$$

where the second equality assumes a constant interest rate. With M independent simulations, the estimator is

$$\hat{C}_0 = \frac{1}{M} \sum_{i=1}^{M} C_{0,i}.$$

Because the payoffs are sampled randomly, the estimated price is itself a random quantity. Its standard error is estimated from the sample standard deviation of the discounted payoffs:

$$\mathrm{SE}\left(\hat{C}_0\right) = \frac{s_C}{\sqrt{M}}, \qquad s_C^2 = \frac{1}{M-1} \sum_{i=1}^{M} \left(C_{0,i} - \hat{C}_0\right)^2.$$

The standard error measures sampling uncertainty in the price estimate. It should be distinguished from the dispersion of individual stock prices or option payoffs. For independent samples with finite variance, increasing M reduces the standard error at a rate proportional to $M^{-1/2}$.

European calls in the Black Scholes model

A European call can be exercised only at its maturity date. Its terminal payoff is

$$C_T = \max(S_T - K, 0),$$

where S_T is the underlying price at maturity and K is the strike. The option pays a positive amount when the underlying price exceeds the strike and expires without a payoff otherwise. An American option, in contrast, permits exercise before maturity.

The Black–Scholes framework provides analytical prices for European calls and puts under a geometric Brownian motion model with constant volatility and a constant risk-free rate. The associated pricing equation can be transformed into a heat equation, providing a connection with diffusion methods in physics. The analytical value is used here as a benchmark for the simulation estimate [1,2].

Under risk-neutral dynamics, the logarithm $x_t = \ln S_t$ satisfies

$$dx_t = \nu\, dt + \sigma\, dW_t, \qquad \nu = r - \frac{\sigma^2}{2}.$$

For a time interval Δt,

$$x_{t+\Delta t} = x_t + \nu\Delta t + \sigma\Delta W_t, \qquad \Delta W_t \sim \mathcal{N}(0, \Delta t).$$

Writing $\Delta W_t = \sqrt{\Delta t}\,\varepsilon$ with $\varepsilon \sim \mathcal{N}(0,1)$ gives the stock-price update

$$S_{t+\Delta t} = S_t \exp\left[\left(r - \frac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t}\,\varepsilon\right].$$

With constant coefficients, this exponential update samples the geometric Brownian motion exactly at the chosen time points. The log price is normally distributed; the stock price itself is lognormally distributed. The factor e^{-rT} discounts the payoff and should not be confused with the interest rate r.

Loop based and vectorized simulation

Source implementation 1.1.3. The option-pricing example uses the inputs S, K, vol, r, N, M, market_value, and T. These represent the current stock price, strike, annualized volatility, annual risk-free rate, number of time steps, number of paths, observed market option price, and time to maturity in years. The original implementation calculates maturity from specified calendar dates.

In the loop-based implementation, each path is advanced through N time steps. The log price is updated using a deterministic term nudt and a random term scaled by volsdt. The terminal call payoff is calculated and accumulated across all paths. Discounting the average payoff produces C0, and the sample variability is used to calculate SE.

The vectorized implementation generates arrays of standard normal samples for all paths and time steps. NumPy operations calculate successive log-price increments, delta_lnSt, and their cumulative sum, lnSt. Terminal stock prices and option payoffs are then evaluated as arrays. The mean and standard error are calculated using the same statistical estimator as in the loop-based version. Vectorization changes the computational implementation rather than the underlying pricing model.

The output includes the estimated call price and its standard error, a normal approximation centered on the price estimate, and simulated stock-price paths. The normal curve represents an approximation to estimator uncertainty, rather than the distribution of individual option payoffs, which need not be normal.

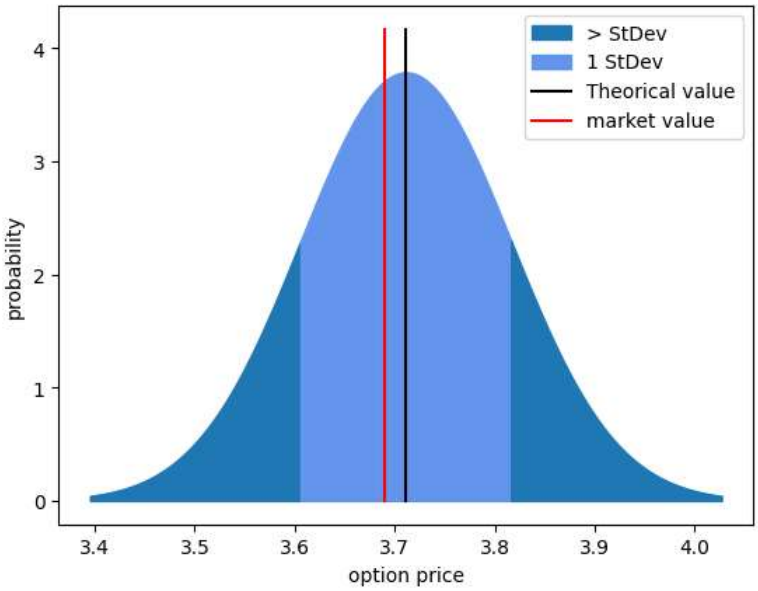


Figure 3. Normal approximation around the simulated option-price estimate in source implementation 1.1.3. The shading distinguishes the central one-standard-deviation region. Black and red lines mark the theoretical and market reference values.

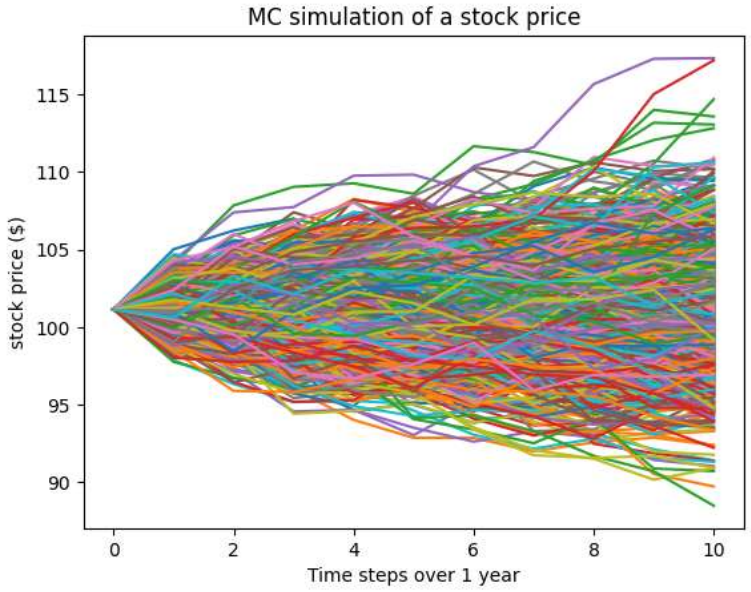


Figure 4. Simulated underlying-stock trajectories over the one-year window described in the source. The horizontal axis records discrete time-step indices within that window.

Variance reduction methods

Antithetic variates

Antithetic variates exploit paired random inputs to reduce the variance of a Monte Carlo estimator [1]. Each standard normal input ε is paired with $-\varepsilon$. Both have the same marginal distribution, but their dependence can generate negatively correlated payoffs. Averaging those payoffs can reduce sampling variability.

The source introduces the method through an analogy with option hedging. A put typically has a negative delta, meaning that its value decreases as the underlying price increases. Delta is the derivative of option value with respect to the underlying price. This sign should not be interpreted as a perfect negative correlation between the stock price and put delta; in the Black–Scholes model, a put's delta generally becomes less negative as the underlying price rises.

For the antithetic construction, two simulated assets start with the same price and use equal drift and volatility coefficients, but opposite Brownian shocks:

$$dS_{1,t} = rS_{1,t}\,dt + \sigma S_{1,t}\,dW_t,$$

$$dS_{2,t} = rS_{2,t}\,dt - \sigma S_{2,t}\,dW_t.$$

The Brownian shocks are perfectly negatively correlated. This does not imply that the resulting nonlinear asset-price levels have correlation exactly equal to minus one. The two processes have the same marginal distribution, so otherwise identical options on them have the same model value.

For a European call, paired terminal payoffs can be written as

$$C_{T,i}^{\pm} = \max\left(S_0 e^{\nu T \pm \sigma\sqrt{T}\,\varepsilon_i} - K, 0\right),$$

and their average is

$$A_i = \frac{C_{T,i}^{+} + C_{T,i}^{-}}{2}, \qquad \hat{C}_0^{\text{anti}} = e^{-rT}\frac{1}{M}\sum_{i=1}^{M} A_i.$$

The intuition is that an input producing a relatively large call payoff on one trajectory tends to produce a smaller payoff on its partner. The variance of the pair average depends on the payoff covariance:

$$\text{Var}(A_i) = \frac{1}{4}\left[\text{Var}\left(C_{T,i}^{+}\right) + \text{Var}\left(C_{T,i}^{-}\right) + 2\text{Cov}\left(C_{T,i}^{+}, C_{T,i}^{-}\right)\right].$$

Negative covariance makes the average less variable than an average of two independent payoffs with the same marginal variance. Generating a paired input also costs less than drawing two separate normal samples, and the mean of each input pair is exactly zero. The method targets variance, rather than correcting bias in the simulation model.

Source implementation 1.1.4. The antithetic example imports NumPy, SciPy, and Matplotlib; specifies the option and simulation inputs; and calculates the constant drift and volatility terms in advance. The sums of payoffs and squared payoffs are accumulated for the price and standard-error estimates. Both loop-based and vectorized versions generate the paired paths, evaluate their terminal payoffs, and compare the result with a simulation without antithetic sampling.

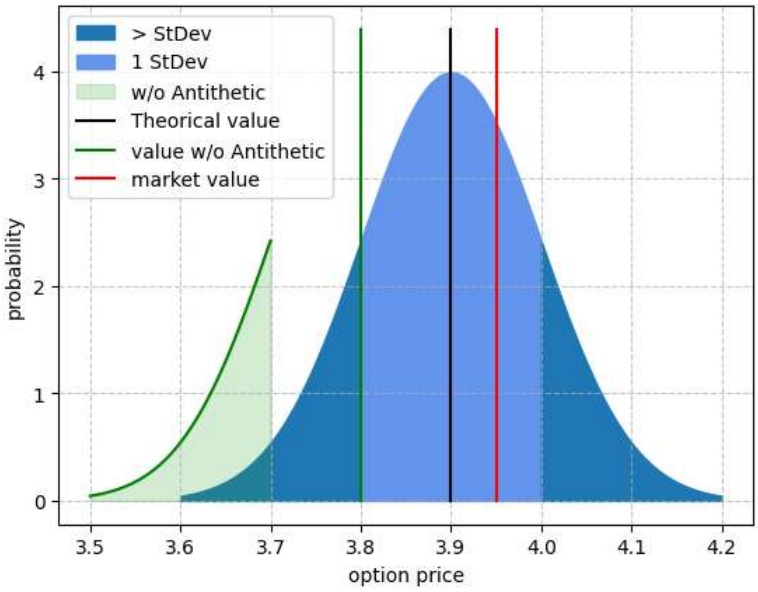


Figure 5. Source comparison of ordinary and antithetic Monte Carlo estimates. The green curve and line are labelled "without antithetic" in the original figure; black and red lines mark the theoretical and market values. The curve widths illustrate the uncertainty approximations used in that example.

The plotted comparison should be interpreted using its own labels. The green reference is the result without antithetic sampling, rather than the antithetic estimate. Conclusions about efficiency must also account for the number of independent payoff pairs and the computational effort used to generate them.

Control variates and discrete delta hedging

Control variates use an auxiliary random variable with known expectation and a strong relationship to the quantity being estimated. Subtracting a suitable centered control can reduce the variance without changing the target expectation [1,2]. Delta hedging motivates one such construction for option valuation.

A delta-neutral portfolio offsets the local sensitivity of an option with a position in its underlying asset. In continuous-time theory, the hedge is adjusted continuously. A practical or numerical implementation rebalances at discrete intervals. Between rebalancing times, the underlying price and option sensitivities change, creating a residual hedge error. Shorter intervals can reduce this error, although actual trading would also involve transaction costs. Here, the hedge is used as a numerical control, rather than as an executed trading strategy.

Let $0 = t_0 < t_1 < \cdots < t_N = T$, and let $\Delta_i = \partial C(t_i, S_{t_i}) / \partial S$ denote the option delta at the beginning of interval i. With zero initial stock holdings before the first hedge and liquidation at maturity, the terminal value of the financed hedge can be written as

$$H_T = C_0 e^{rT} - \sum_{i=0}^{N} (\Delta_i - \Delta_{i-1}) S_{t_i} e^{r(T-t_i)},$$

where $\Delta_{-1} = 0$ and $\Delta_N = 0$ encode initialization and liquidation. Rearranging the rebalancing cash flows gives

$$H_T = C_0 e^{rT} + cv_1,$$

with the delta-based control

$$cv_1 = \sum_{i=0}^{N-1} \Delta_i \left(S_{t_{i+1}} - S_{t_i} e^{r\Delta t_i}\right) e^{r(T-t_{i+1})}.$$

Here $\Delta t_i = t_{i+1} - t_i$. Under the risk-neutral geometric Brownian motion model,

$$\mathbb{E}^{\mathbb{Q}}\left[S_{t_{i+1}} \mid \mathcal{F}_{t_i}\right] = S_{t_i} e^{r\Delta t_i}.$$

The stock-price increment after removal of its conditional mean is therefore centered. Provided Δ_i depends only on information available at t_i, the control has zero expectation. If $\eta = H_T - C_T$ denotes the residual hedge error, the relation becomes

$$C_0 e^{rT} = C_T - cv_1 + \eta.$$

These expressions present the source's rebalancing expansion in compact form and make the cash-flow signs and conditional expectations explicit. In a sufficiently accurate hedge, subtracting cv_1 removes much of the variation in the terminal payoff.

Gamma based and multiple control variates

The same principle can be extended to a control based on gamma, the second derivative of option value with respect to the underlying price. Let $\Gamma_i = \partial^2 C\left(t_i, S_{t_i}\right) / \partial S^2$. A centered gamma-based control is

$$cv_2 = \sum_{i=0}^{N-1} \Gamma_i \left[\left(S_{t_{i+1}} - S_{t_i}\right)^2 - m_{2,i}\right] e^{r(T-t_{i+1})},$$

where the conditional second moment of the price change is

$$m_{2,i} = S_{t_i}^2 \left[e^{(2r+\sigma^2)\Delta t_i} - 2e^{r\Delta t_i} + 1\right].$$

Because the squared increment is centered by its conditional expectation, this control also has zero expectation under the stated model. A constant normalization factor, such as the one-half appearing in a Taylor expansion, can be incorporated into the control coefficient.

For J controls, a regression representation of the terminal payoff is

$$C_T = \beta_0 + \sum_{j=1}^{J} \beta_j \, cv_j + \eta, \qquad \beta_0 = C_0 e^{rT}.$$

The coefficients describe the linear relationship between the payoff and the controls, and the residual contains variation not explained by them. In the hedging interpretation, discretization and approximations to delta or gamma contribute to that residual. The corresponding adjusted payoff is

$$C_T^{\text{adj}} = C_T - \sum_{j=1}^{J} \beta_j \, cv_j.$$

The coefficients can be estimated from a preliminary sample using least-squares regression:

$$\hat{\beta} = (X^{\top} X)^{-1} X^{\top} Y,$$

when the displayed inverse exists. The design matrix contains the control observations and an intercept, and Y contains terminal payoffs. Using a separate preliminary sample to estimate the coefficients avoids

the finite-sample dependence introduced when the same observations determine both the coefficients and the final price estimate.

The source also uses an additive-adjustment convention, $C_T^{\text{adj}} = C_T + b_1 cv_1$. In that convention the delta-hedge choice is $b_1 = -1$. This is equivalent to a coefficient of $+1$ in the subtraction convention above. Consistent signs are essential when implementing the control.

Delta control implementation

Source implementation 1.1.5. The described script imports math, NumPy, pandas, datetime, SciPy's statistics tools, Matplotlib, and py_vollib routines for Black–Scholes calculations and implied volatility. The initial inputs specify the stock price, strike, volatility, risk-free rate, maturity, number of time steps, number of paths, and market reference price.

The quantities dt, nudt, volsdt, and the one-step growth factor erdt are calculated before the simulation. For each path, the script advances the underlying stock price and evaluates the call delta using the current stock price and remaining maturity. The control accumulates the difference between the next stock price and its conditional expected value, weighted by the current delta. The terminal call payoff is combined with the signed control adjustment, averaged over simulations, and discounted by exp(-r*T).

The script reports the estimated option price and standard error, then plots the approximation to estimator uncertainty against theoretical, market, and uncontrolled reference values. The source labels this implementation as a delta-based method; an isolated reference to a gamma control in its description is inconsistent with the delta calculation that follows.

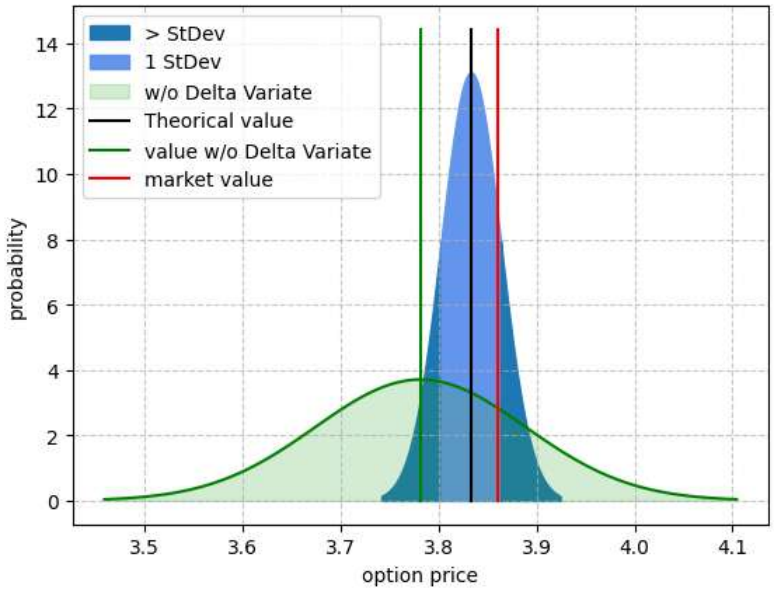


Figure 6. Source comparison of option-price estimates with and without the delta-based control. The green curve and line are labelled "without delta variate"; the narrower blue distribution represents the adjusted estimate. Black and red lines identify the theoretical and market values.

In this individual example, the adjusted curve is narrower than the uncontrolled curve, indicating a smaller plotted standard error. The figure also places the adjusted estimate closer to the theoretical reference. This visual result should be assessed separately from the combined benchmark below, which reports a different ordering of standard errors.

Combining antithetic and control variates

Source implementation 1.1.6. The comparison extends the earlier scripts by recording runtime and implementing combinations of antithetic sampling with delta and gamma controls. Both members of each antithetic pair have a call payoff and associated controls.

For path $a \in \{1,2\}$, the delta and gamma controls use the path-specific sensitivities $\Delta_{a,i}$ and $\Gamma_{a,i}$:

$$cv_{1,a} = \sum_{i=0}^{N-1} \Delta_{a,i} \left(S_{a,t_{i+1}} - S_{a,t_i} e^{r\Delta t_i}\right) e^{r(T-t_{i+1})},$$

$$cv_{2,a} = \sum_{i=0}^{N-1} \Gamma_{a,i} \left[\left(S_{a,t_{i+1}} - S_{a,t_i}\right)^2 - m_{2,a,i}\right] e^{r(T-t_{i+1})},$$

where

$$m_{2,a,i} = S_{a,t_i}^2 \left[e^{(2r+\sigma^2)\Delta t_i} - 2e^{r\Delta t_i} + 1\right].$$

The pair-averaged controls are $\overline{cv}_j = \left(cv_{j,1} + cv_{j,2}\right)/2$. Combining the paired payoffs and delta control gives

$$A_i^{\Delta} = \frac{C_{T,i}^{+} + C_{T,i}^{-}}{2} - \beta_1 \overline{cv}_{1,i}.$$

Including gamma gives

$$A_i^{\Delta,\Gamma} = \frac{C_{T,i}^{+} + C_{T,i}^{-}}{2} - \beta_1 \overline{cv}_{1,i} - \beta_2 \overline{cv}_{2,i}.$$

These averages are discounted and averaged over independent pairs to estimate the initial option value. The source's formulas contain repeated uses of the control coefficients and mixed notation for sensitivity and price change. The expressions above apply each coefficient once and distinguish delta from ΔS.

Reported comparison of precision and runtime

The original benchmark compares plain Monte Carlo, antithetic sampling, delta and gamma controls, the antithetic–delta combination, and the combination of all controls. Table 1 reproduces the numerical values displayed in the source's Figure 7 without modifying them.

Table 1. Reported benchmark from source implementation 1.1.6. "SE reduction multiple" and relative computation time are the column labels used in the original output.

Method	Standard error	SE reduction multiple	Relative runtime
Plain Monte Carlo	0.0007	1.00	1.00
Antithetic variates	0.0287	0.03	0.59
Delta control	0.0080	0.09	2.23
Gamma control	0.1038	0.01	1.79
Antithetic and delta	0.0059	0.13	2.87

Method	Standard error	SE reduction multiple	Relative runtime
All combined variates	0.0014	0.55	2.44

	Standard Error (SE)	SE Reduction Multiple	Relative Computation Time
Simple estimate	0.0007	1.00	1.00
with antithetic variate	0.0287	0.03	0.59
with delta-based control variate	0.0080	0.09	2.23
with gamma-based control variate	0.1038	0.01	1.79
with antithetic and delta variates	0.0059	0.13	2.87
with all combined variates	0.0014	0.55	2.44

Figure 7. Original benchmark output retained for comparison with Table 1. The displayed standard errors and relative runtimes have not been recalculated.

Within this particular table, the plain estimator has the smallest reported standard error. The delta control performs better than antithetic sampling according to the displayed error ratio, and the combined methods improve that ratio relative to the individual variance-reduction methods. However, all the reported ratios remain below the plain-method reference value of one. The values therefore do not demonstrate an overall reduction in standard error relative to plain Monte Carlo for this benchmark.

The reported antithetic runtime is lower than the baseline, whereas the control-based and combined methods take approximately 1.79 to 2.87 times the baseline runtime. Their additional calculations have a measurable cost. A fair comparison requires matching the simulated inputs, the number of independent observations or pairs, the standard-error definition, and the computational budget. The apparent conflict between this table and the uncertainty plots in Figures 5 and 6 cannot be resolved from the report alone.

Stochastic calculus for financial models

Brownian motion

Brownian motion is a basic stochastic process underlying continuous-time financial models [2,4]. A standard Brownian motion $\{W_t\}_{t\in[0,T]}$ is defined on a filtered probability space $(\Omega, \mathcal{F}, \mathbb{P}, \{\mathcal{F}_t\})$, with $W_0 = 0$. It has continuous sample paths and independent Gaussian increments:

$$W_t - W_s \sim \mathcal{N}(0, t-s), \qquad 0 \le s < t \le T.$$

Brownian motion is both a Markov process and a martingale with respect to its natural filtration. These are distinct properties: the Markov property concerns dependence on the present state, while the martingale property concerns conditional expectations. Its quadratic variation accumulates at unit rate, $[W]_t = t$. This nonzero quadratic variation is one reason stochastic calculus differs from ordinary calculus.

Symmetric random walks

A symmetric random walk can be constructed from independent fair coin tosses. Let $X_j = +1$ for a head and $X_j = -1$ for a tail, with each outcome having probability one-half. Define

$$M_0 = 0, \qquad M_k = \sum_{j=1}^{k} X_j.$$

For indices $0 = k_0 < k_1 < \cdots < k_m$, the increments $M_{k_{i+1}} - M_{k_i}$ are independent and satisfy

$$\mathbb{E}\left[M_{k_{i+1}} - M_{k_i}\right] = 0, \qquad \mathrm{Var}\left(M_{k_{i+1}} - M_{k_i}\right) = k_{i+1} - k_i.$$

Thus the ensemble variance grows at a rate of one per discrete time step. The process is also a martingale:

$$\mathbb{E}[M_l \mid \mathcal{F}_k] = M_k, \qquad k < l.$$

The conditional expectation of a later position equals the current position. Since every step has squared length one, the discrete quadratic variation is

$$[M]_k = \sum_{j=1}^{k} \left(M_j - M_{j-1}\right)^2 = k.$$

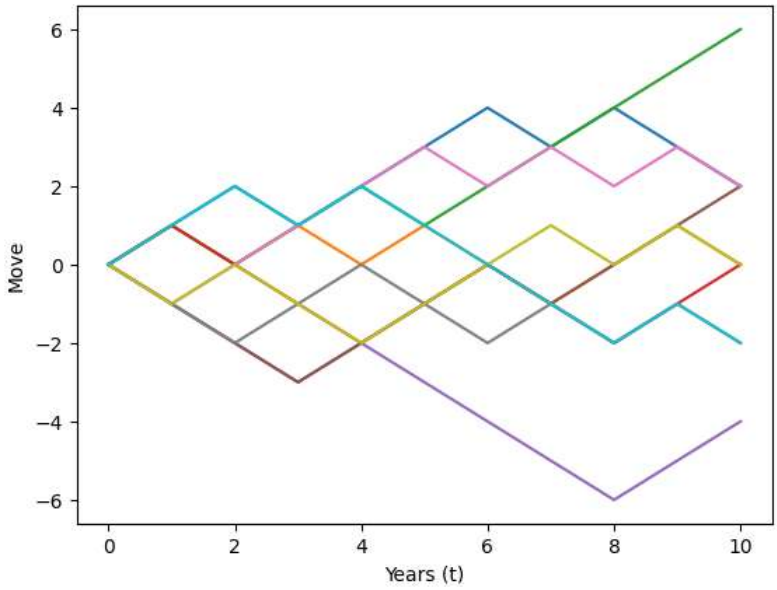


Figure 8. Symmetric random-walk trajectories over the ten-year time axis used in source implementation 1.2.1.

Scaled random walks and the Brownian limit

To approximate Brownian motion, the number of steps per unit time is increased while the step size is reduced. A scaled random walk is

$$W^{(n)}(t) = \frac{1}{\sqrt{n}} M_{\lfloor nt \rfloor}.$$

At fixed time t, its variance approaches t. In the limit of large n, its terminal distribution converges to a normal distribution:

$$W^{(n)}(t) \xrightarrow{\mathrm{d}} \mathcal{N}(0, t).$$

This statement is a convergence-in-distribution result. With appropriate interpolation, the sequence of processes also provides the familiar random-walk approximation to Brownian motion.

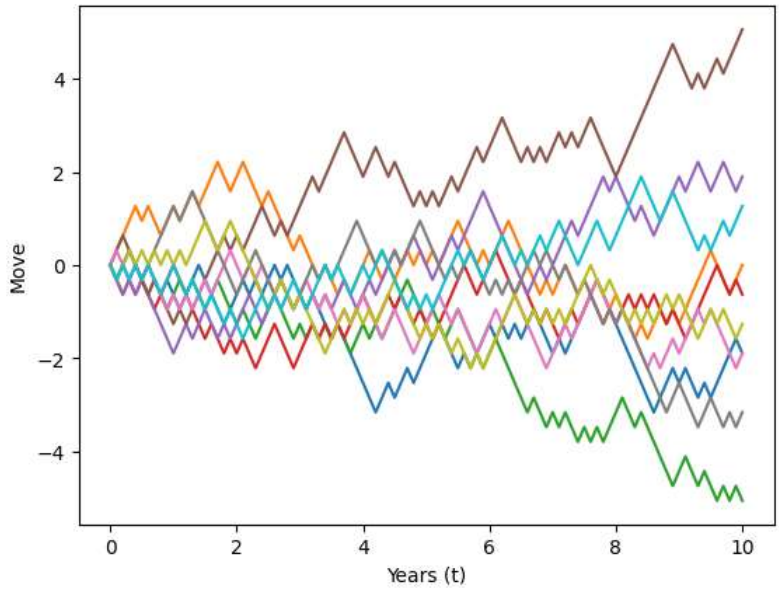


Figure 9. Scaled symmetric random-walk trajectories over a ten-year window. The step size is reduced as the temporal resolution increases.

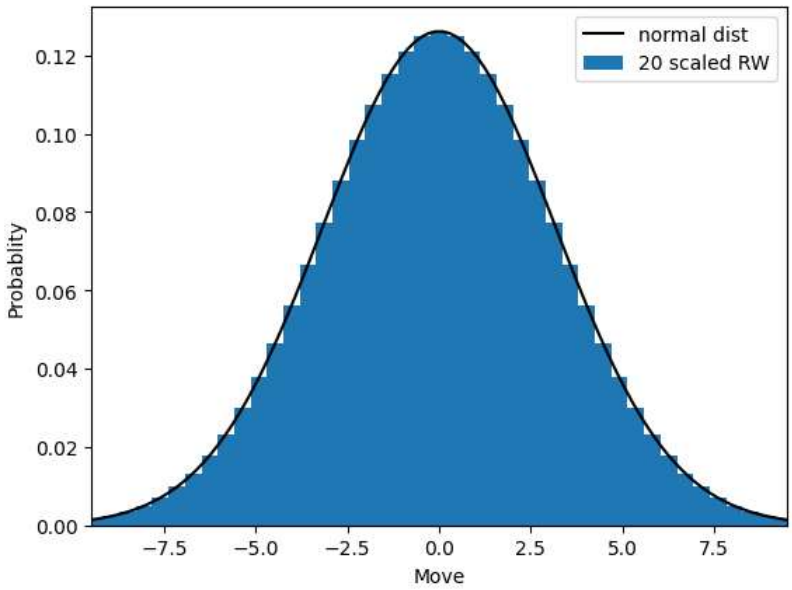


Figure 10. Distribution of a scaled symmetric random walk compared with a normal approximation. The source uses the comparison to illustrate convergence of the binomial endpoint distribution toward a Gaussian distribution.

Random walk and Brownian simulation procedure

Source implementation 1.2.1. The described Python workflow imports math, itertools, NumPy, SciPy statistics, and Matplotlib. It first constructs simple random walks by drawing increments from $\{-1, +1\}$ and applying the cumulative-sum operation. The initial example uses ten paths over a ten-unit time interval, and the paths are plotted to visualize the accumulated displacements.

Two functions are described: quadratic_variation sums squared differences between successive positions, while variance calculates the variance of the supplied array. They are applied to selected paths from the small example and from a much larger simulation described in the source as containing

10,000,000 realizations. The array variance along a single path should be distinguished from the theoretical variance across independent realizations at a fixed time.

The scaled-walk simulation uses tn steps over a horizon t, with increments divided by $\sqrt{n}$. The accumulated paths are plotted against their rescaled time coordinates. A combinatorial calculation using nCr evaluates the number of sequences producing each terminal position. The resulting binomial endpoint distribution is compared with a normal curve.

The Brownian simulation uses ten trajectories, a ten-unit horizon, and a temporal resolution parameter $n = 100$. Independent Gaussian increments with variance equal to the time step are generated and accumulated. The same quadratic-variation and variance calculations are then applied. Together, these examples connect discrete random walks, their scaled form, and Gaussian-increment simulations of Brownian motion.

Geometric Brownian motion

Geometric Brownian motion (GBM) is a multiplicative stochastic model for a positive asset price. For $S_0 > 0$, its dynamics are

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t,$$

where μ is the drift and σ is the volatility [2,3]. Applying Itô's formula to the logarithm gives

$$\ln S_t = \ln S_0 + \int_0^t \left(\mu - \frac{\sigma^2}{2}\right) ds + \int_0^t \sigma\ dW_s.$$

With constant coefficients,

$$\ln S_t = \ln S_0 + \left(\mu - \frac{\sigma^2}{2}\right) t + \sigma W_t,$$

so that

$$\ln S_t \sim \mathcal{N}\left(\ln S_0 + \left(\mu - \frac{\sigma^2}{2}\right) t, \sigma^2 t\right).$$

The explicit price process is

$$S_t = S_0 \exp\left[\left(\mu - \frac{\sigma^2}{2}\right) t + \sigma W_t\right].$$

Two equivalent simulation procedures are described. One generates successive exponential price multipliers and takes their cumulative product. The other generates normal log-price increments, accumulates them, and exponentiates the result. In either case, the stochastic model produces positive, lognormally distributed stock prices.

Geometric Brownian motion implementation

Source implementation 1.2.2. The GBM example uses NumPy for the simulation and Matplotlib for visualization. The inputs are the drift mu, number of time steps n, total horizon T, number of paths M, initial stock price S0, and volatility sigma. The time step is calculated as dt = T/n.

Independent normal samples are transformed into exponential factors using the drift and volatility terms. The cumulative-product operation, cumprod, combines those factors along each path. An initial row of ones is included so that multiplication by S0 places every trajectory at the same starting price. A

compatible time array is created with linspace and broadcast to match the simulated price array for plotting.

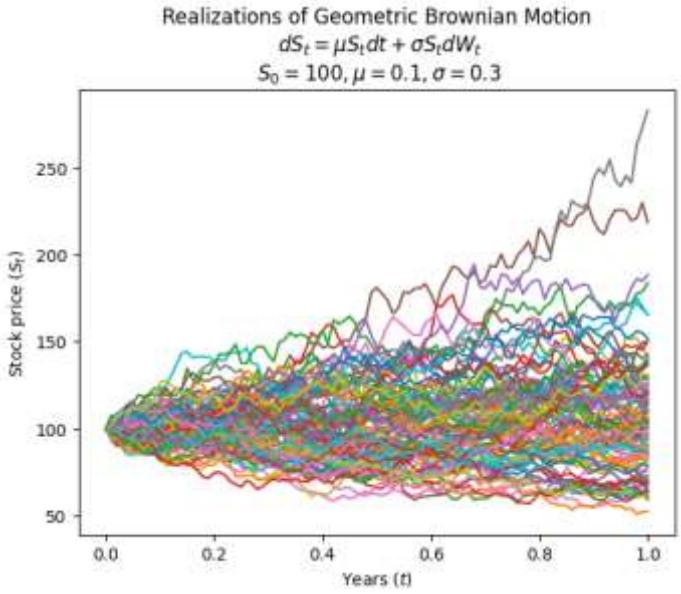


Figure 11. Simulated GBM trajectories over one year, retained from the source. The figure specifies $S_0 = 100$, $\mu = 0.1$, and $\sigma = 0.3$.

The paths illustrate the range of stock prices produced by the chosen model and parameters. They represent individual simulated asset trajectories; a portfolio model would additionally require asset weights and a dependence structure. The drift μ in this illustration should be distinguished from the risk-free drift r used for risk-neutral option valuation.

Binomial option pricing and computational efficiency

The binomial model and absence of arbitrage

The binomial asset-pricing model represents changes in an underlying price through successive upward and downward moves. It provides a discrete framework for option valuation and a direct connection between no-arbitrage reasoning and risk-neutral probabilities [1,3].

Arbitrage is an opportunity to obtain a nonnegative terminal outcome with a positive chance of gain without a positive initial cost. The source distinguishes two forms. In the first,

$$V_0 = 0, \qquad \mathbb{P}(V_T \geq 0) = 1, \qquad \mathbb{P}(V_T > 0) > 0.$$

In the second,

$$V_0 < 0, \qquad \mathbb{P}(V_T \geq 0) = 1.$$

The first begins with zero initial investment and permits a gain without a loss. The second provides an initial cash inflow while guaranteeing a nonnegative terminal value. The pricing model excludes such opportunities under its assumptions.

Tree representation and backward valuation

A recombining tree is indexed by nodes (i, j), where i is the time-step index and j is the number of upward moves. The underlying price at a node is

$$S_{i,j} = S_0 u^j d^{\,i-j}, \qquad j = 0,1, \dots, i.$$

Here u and d are the upward and downward multipliers. At maturity, a European call has value

$$C_{N,j} = \max\big(S_{N,j} - K, 0\big).$$

The implementation calculates the time step, risk-neutral probability, and discount factor before applying backward induction. With $\Delta t = T/N$, these quantities are

$$q = \frac{e^{r\Delta t} - d}{u - d}, \qquad D = e^{-r\Delta t},$$

and the valuation recursion is

$$C_{i,j} = D\big[qC_{i+1,j+1} + (1-q)C_{i+1,j}\big].$$

The risk-neutral probability is valid when the no-arbitrage condition $d < e^{r\Delta t} < u$ is satisfied. The call recursion proceeds from terminal payoffs to the initial node. The described routines also accept a put flag, for which the terminal payoff is $\max\big(K - S_{N,j}, 0\big)$.

Loop-based and vectorized implementations

Source implementation 1.3. Two Python functions, binomial_tree_slow and binomial_tree_fast, implement the same backward-valuation procedure. The first loops explicitly over nodes j at each time step i. The second performs the node calculations with NumPy arrays, retaining the backward progression through time.

The inputs are the initial price S0, strike K, maturity T, annual risk-free rate r, number of time steps N, upward multiplier u, and option-type flag opttype. The downward multiplier is calculated as d = 1/u. The option flag selects a call (c) or put (p). Both functions initialize terminal asset prices and payoffs, calculate the one-step risk-neutral probability and discount factor, and work backward to the initial value.

A timing decorator measures elapsed runtime by recording times before and after each function call. The wraps utility from functools preserves the metadata of the wrapped function. NumPy supplies the numerical operations, and the time module supplies the runtime measurement.

The comparison changes the number of time steps while keeping the upward and downward multipliers fixed. As noted in the source, changes in the resulting option values should not be interpreted as convergence to a common continuous-time model, because changing N while fixing u and d also changes the implied price dynamics. The exercise is used to compare computational implementation costs.

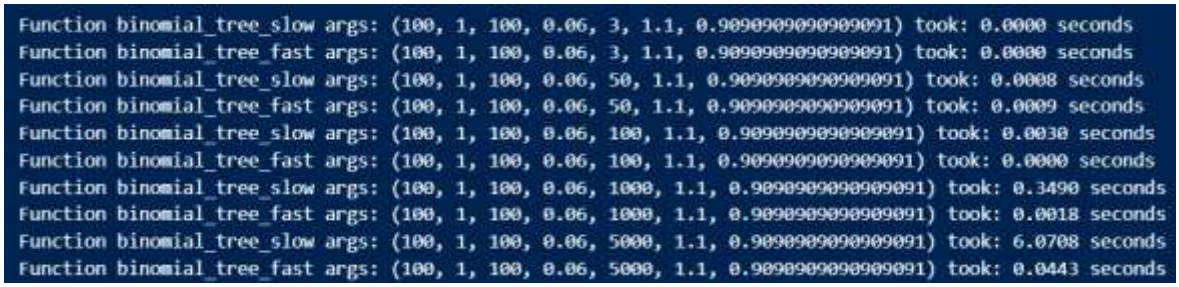


Figure 12. Recorded runtimes for loop-based and vectorized binomial valuation. This output was labelled Figure 13 in the source; the numbering is made consecutive in the English version.

The source output reports approximately 0.3490 seconds for the loop-based method and 0.0018 seconds for the vectorized method at $N = 1{,}000$. At $N = 5{,}000$, the reported runtimes are 6.0708 and 0.0443 seconds, respectively. These measurements show a substantial speed advantage for the vectorized implementation in the recorded examples. They are individual timing observations; the output does not report repeated measurements or uncertainty in runtime.

Concluding discussion

The examples connect methods from computational physics with portfolio risk analysis and derivative valuation. Cholesky decomposition enables correlated Gaussian return simulation, while Monte Carlo sampling provides portfolio-value scenarios and estimates of lower-tail risk. For options, risk-neutral dynamics and discounted expected payoffs define the valuation problem. Antithetic and control variates provide mechanisms for reducing sampling variability, and stochastic-process models explain the random trajectories used in these calculations.

The practical examples also show the importance of distinguishing model assumptions from empirical forecasting claims. Simulated paths depend on the chosen return distribution, drift, volatility, dependence structure, and time horizon. VaR and CVaR describe different aspects of a modelled loss distribution, and standard errors describe uncertainty in an estimator rather than uncertainty in the model itself.

The runtime comparison supports the computational benefit of NumPy vectorization in the recorded binomial examples. The combined variance-reduction benchmark, however, does not establish that the adjusted estimators outperform the plain estimator under comparable conditions. Resolving that inconsistency requires verification of the underlying implementations and simulation settings. The report therefore illustrates both the utility of these numerical methods and the need for consistent definitions and controlled comparisons when assessing their performance.

References

[1] Clewlow, L., and Strickland, C. (1998). *Implementing Derivative Models*. Wiley.

[2] Shreve, S. E. (2004). *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer. https://doi.org/10.1007/978-1-4757-4296-1

[3] Shreve, S. E. (2004). *Stochastic Calculus for Finance I: The Binomial Asset Pricing Model*. Springer. https://doi.org/10.1007/978-0-387-22527-2

[4] Jacobs, K. (2010). *Stochastic Processes for Physicists: Understanding Noisy Systems*. Cambridge University Press. https://doi.org/10.1017/CBO9780511815980

[5] Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering*. Springer. https://doi.org/10.1007/978-0-387-21617-1